

Docker + Spring Boot

实战

开发目标

中小型电商系统

- 从零搭建
- 包含用户登录、商品列表、购物车操作的完整电商核心流程
- 全程基于 Docker 容器化部署
 - 实现快速开发与部署
- 使用 Spring Boot 开发电商系统
 - 遵循 分层架构 设计
 - Spring Boot + MySQL

核心技术栈

- 后端：
 - Spring Boot（核心框架）+ Spring Security（登录认证）+ Spring Data JPA（数据库操作）
- 前端：
 - Thymeleaf（模板引擎，无需单独学前端框架）+ Bootstrap（简易样式）
- 数据库：MySQL（Docker 容器运行，无需本地安装）
- 容器化：Docker + Docker Compose（一键启动应用 + 数据库）

必备工具

- JDK 8+/11 :
 - Java 开发环境
- Maven :
 - 项目构建工具
- Docker :
 - 容器化运行环境
- Docker Compose :
 - 多容器编排工具
- IDE (IDEA/Eclipse) : 代码编写工具

项目整体结构(I)

Spring Boot 分层开发（实体-仓库-服务-控制器）

- 实体层（Entity）：映射数据库表（如用户表、商品表）
 - `com.example.ecommerce.entity`
 - JPA 注解（`@Entity`、`@Table`）
- 仓库层（Repository）：数据库操作（增删改查）
 - `com.example.ecommerce.repository`
 - Spring Data JPA（`JpaRepository`）
- 服务层（Service）：业务逻辑处理（如加入购物车、结算）
 - `com.example.ecommerce.service`
 - `@Service` 注解

项目整体结构(II)

Spring Boot 分层开发（实体-仓库-服务-控制器）

- 控制器层（Controller）：接收前端请求、返回页面 / 数据
 - `com.example.ecommerce.controller`
 - `@Controller`、`@GetMapping` 等
- 配置层（Config）：项目配置（如登录认证、数据库）
 - `com.example.ecommerce.config`
 - Spring Security 配置
- 前端层（Templates）：页面展示（登录页、商品列表）
 - `resources/templates`
 - Thymeleaf + Bootstrap

项目整体结构(III)

这种分层结构是一种经典的 **MVC** 衍生架构，目的是实现代码解耦、职责分离，便于维护和扩展

```
ecommerce-demo/
├── src/
│   ├── main/
│   │   ├── java/com/example/ecommerce/
│   │   │   ├── EcommerceApplication.java // 项目启动类
│   │   │   ├── config/ // 配置类（登录、数据库）
│   │   │   ├── entity/ // 实体类（用户、商品、购物车）
│   │   │   ├── repository/ // 数据库操作层
│   │   │   ├── service/ // 业务逻辑层
│   │   │   └── controller/ // 控制器（接口）
│   │   └── resources/
│   │       ├── static/ // 静态资源（CSS/JS）
│   │       ├── templates/ // 前端页面（Thymeleaf）
│   │       ├── application.properties // 项目配置
│   │       ├── data.sql // 初始化数据（商品/测试用户）
│   │       └── // 测试类（新手可忽略）
│   └── test/
├── Dockerfile // Spring Boot 容器化配置
├── docker-compose.yml // 整合 Spring Boot + MySQL
└── pom.xml // Maven 依赖配置
```

项目整体结构(IV)

这种分层就像 工厂的流水线：

- **实体层：**
 - "原材料规格"（定义数据是什么）；
- **仓库层：**
 - "原材料仓库"（负责存取）；
- **服务层：**
 - "生产车间"（加工处理，实现业务）；
- **控制器层：**
 - "前台窗口"（接收订单、交付产品）。

创建Maven项目

与

配置依赖

创建 Maven 项目

- IDEA 操作: New Project → Maven → 填写 groupId (com.example)、ArtifactId (ecommerce) → 完成

确保 pom.xml 配置如下

```
1<?xml version="1.0" encoding="UTF-8"?>
2<project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven
5      <modelVersion>4.0.0</modelVersion>
6
7      <parent>
8          <groupId>org.springframework.boot</groupId>
9          <artifactId>spring-boot-starter-parent</artifactId>
10         <version>2.7.18</version> <!-- 稳定版, 适配新手 -->
11         <relativePath/>
12     </parent>
13
14     <groupId>com.example</groupId>
15     <artifactId>ecommerce</artifactId>
16     <version>1.0.0</version>
17     <name>ecommerce-demo</name>
18
```

配置依赖(I)

- `spring-boot-starter-web`:
 - 让项目成为 Web 应用
 - 支持接口开发
- `spring-boot-starter-thymeleaf`:
 - 无需前后端分离
 - 直接在后端写前端页面
- `spring-boot-starter-security`:
 - 快速实现用户登录认证
 - 不用自己写登录逻辑

配置依赖(II)

- `spring-boot-starter-data-jpa`:
 - 简化数据库操作
 - 不用写 `SQL` 语句
- `mysql-connector-java`:
 - 连接 `MySQL` 数据库
- `lombok`:
 - 简化实体类代码
 - 比如 `@Data` 注解自动生成 `get/set` 方法

配置项目参数(I)

在 `src/main/resources` 下创建 `application.properties`，内容如下：

```
1# 服务器端口 (Docker 映射用)
2server.port=8080
3
4# 数据库配置 (连接 Docker 中的 MySQL)
5spring.datasource.url=jdbc:mysql://mysql:3306/ecommerce_db?useUnicode=true
6spring.datasource.username=root
7spring.datasource.password=root123
8spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
9
10# JPA 配置 (自动建表、显示SQL)
11spring.jpa.hibernate.ddl-auto=update
12spring.jpa.show-sql=true
13spring.jpa.properties.hibernate.format_sql=true
14spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
15
16# Thymeleaf 配置 (开发时禁用缓存)
17spring.thymeleaf.cache=false
18
```

配置项目参数(II)

关键解释：

- `spring.datasource.url` 中 `mysql` 是 Docker Compose 中 MySQL 容器的名称
 - Docker 内部域名解析
- `spring.jpa.hibernate.ddl-auto=update`：
 - 启动项目时自动创建 / 更新数据库表；
- `spring.thymeleaf.cache=false`：
 - 修改前端页面后无需重启项目
 - 直接刷新浏览器即可

启动类

EcommerceApplication.java

启动类

在 `com.example.ecommerce` 包下创建

`EcommerceApplication.java`:

```
1package com.example.ecommerce;
2
3import org.springframework.boot.SpringApplication;
4import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6@SpringBootApplication
7public class EcommerceApplication {
8    public static void main(String[] args) {
9        SpringApplication.run(EcommerceApplication.class, args);
10    }
11}
```

`@SpringBootApplication`:

- `Spring Boot` 核心注解，自动配置、扫描组件
- 新手无需理解细节，加上即可

实体类 (Entity)

(用户、商品、购物车)

实体类 (I)

- 数据模型映射
- 角色：数据库表的 "镜像", 负责定义数据结构。
- 作用：
 - 用 **Java** 类映射数据库中的表, 每个实体类对应一张表, 类中的字段对应表中的列。
 - 通过 **JPA** 注解 (如 **@Entity**、**@Table**、**@Id**) 定义表名、主键、字段约束 (如唯一、非空) 等。
 - 存储业务数据的基本结构, 不包含复杂逻辑, 仅保留数据的 "形"。

实体类 (II) :

在 `com.example.ecommerce.entity` 包下创建用户实体
(`User.java`)

```
1package com.example.ecommerce.entity;
2
3import lombok.Data;
4import javax.persistence.*;
5import java.util.Set;
6
7@Data
8@Entity
9@Table(name = "users") // 数据库表名
10public class User {
11    @Id
12    @GeneratedValue(strategy = GenerationType.IDENTITY) // 自增ID
13    private Long id;
14
15    @Column(unique = true, nullable = false) // 用户名唯一、非空
16    private String username;
17
18    @Column(nullable = false)
19    private String password; // 密码 (Spring Security 会加密)
```

实体类（III）：

在 `com.example.ecommerce.entity` 包下创建商品实体
(`Product.java`)

```
1package com.example.ecommerce.entity;
2
3import lombok.Data;
4import javax.persistence.*;
5import java.math.BigDecimal;
6
7@Data
8@Entity
9@Table(name = "products")
10public class Product {
11    @Id
12    @GeneratedValue(strategy = GenerationType.IDENTITY)
13    private Long id;
14
15    @Column(nullable = false)
16    private String name; // 商品名称
17
18    private String description; // 商品描述
19}
```

实体类 (IV) :

购物车项实体 (CartItem.java)

```
1package com.example.ecommerce.entity;
2
3import lombok.Data;
4import javax.persistence.*;
5
6@Data
7@Entity
8@Table(name = "cart_items")
9public class CartItem {
10    @Id
11    @GeneratedValue(strategy = GenerationType.IDENTITY)
12    private Long id;
13
14    // 关联用户 (多对一)
15    @ManyToOne
16    @JoinColumn(name = "user_id", nullable = false)
17    private User user;
18
19    // 关联商品 (多对一)
```

实体类解释

- `@Entity`: 标记为 JPA 实体
 - 对应数据库表;
- `@Table`: 指定数据库表名;
- `@Id + @GeneratedValue`: 主键 + 自增
- `@OneToMany/@ManyToOne`: 表之间的关联关系
 - 用户-购物车项 是一对多
 - 购物车项-商品 是多对一
- `@Data (Lombok)`: 自动生成 `get/set/toString` 等方法
 - 简化代码

数据库操作层

(Repository)

数据库操作层 (I)

- 又可称为**仓库层**：数据访问接口
- **角色**：数据库操作的 "执行者"，负责数据的增删改查。
- **作用**：
 - 基于 `Spring Data JPA`，通过接口继承 `JpaRepository`，无需手写 `SQL` 即可实现基本 `CRUD` 操作（如 `save()`、`findById()`、`findAll()`）。
 - 支持自定义查询方法（如 `findByUsername`），`JPA` 会根据方法名自动生成 `SQL`，简化数据访问逻辑。
 - 隔离数据库操作细节，使上层（服务层）无需关注数据存储的具体实现。

数据库操作层 (II)

在 `com.example.ecommerce.repository` 包下创建

`UserRepository.java`

```
1package com.example.ecommerce.repository;
2
3import com.example.ecommerce.entity.User;
4import org.springframework.data.jpa.repository.JpaRepository;
5import java.util.Optional;
6
7// JpaRepository<实体类, 主键类型>: 内置增删改查方法
8public interface UserRepository extends JpaRepository<User, Long> {
9    // 根据用户名查询用户 (登录时用)
10    Optional<User> findByUsername(String username);
```

在 `com.example.ecommerce.repository` 包下创建

`ProductRepository.java`

```
1package com.example.ecommerce.repository;
2
3import com.example.ecommerce.entity.Product;
4import org.springframework.data.jpa.repository.JpaRepository;
5
6public interface ProductRepository extends JpaRepository<Product, Long> {
7    // 无需写额外方法, JpaRepository 已有查询所有、根据ID查询等方法
8}
```

数据库操作层 (III)

在 `com.example.ecommerce.repository` 包下创建
`CartItemRepository.java`

```
1package com.example.ecommerce.repository;
2
3import com.example.ecommerce.entity.CartItem;
4import com.example.ecommerce.entity.User;
5import org.springframework.data.jpa.repository.JpaRepository;
6import java.util.List;
7
8public interface CartItemRepository extends JpaRepository<CartItem, Long> {
9    // 根据用户查询购物车项
10    List<CartItem> findByUser(User user);
11    // 根据用户和商品查询购物车项（判断是否已加入购物车）
12    CartItem findByUserAndProduct(User user, com.example.ecommerce.entity.P
13}
```

数据库操作层 (IV)

数据库操作层的解释：

- `JpaRepository` 是 Spring Data JPA 提供的通用仓库
 - 内置了 `findAll()`、`findById()`、`save()`、`delete()` 等方法
 - 无需自己写 SQL
- 自定义方法（如 `findByUsername`）
 - JPA 会根据方法名自动生成 SQL
 - 新手无需写复杂查询。

业务逻辑层

(Service)

业务逻辑层 (I)

- 又可称为**服务层**：业务逻辑核心
- **角色**：业务规则的 "决策者"，负责处理核心业务逻辑。
- **作用**：
 - 调用仓库层接口获取或修改数据，结合业务规则进行处理（如购物车中商品数量的增减、订单结算逻辑）。
 - 管理事务（如多表操作时保证原子性，例如创建订单时同时扣减库存和生成订单记录）。
 - 隔离业务逻辑与数据访问、接口交互，使代码模块化，便于复用和测试。

业务逻辑层 (II)

在 `com.example.ecommerce.service` 包下创建
`UserService.java` (用户服务, 登录认证)

```
1package com.example.ecommerce.service;
2
3import com.example.ecommerce.entity.User;
4import com.example.ecommerce.repository.UserRepository;
5import org.springframework.security.core.userdetails.UserDetails;
6import org.springframework.security.core.userdetails.UserDetailsService;
7import org.springframework.security.core.userdetails.UsernameNotFoundException;
8import org.springframework.security.crypto.password.PasswordEncoder;
9import org.springframework.stereotype.Service;
10
11import java.util.Collections;
12
13@Service
14public class UserService implements UserDetailsService {
15    private final UserRepository userRepository;
16    private final PasswordEncoder passwordEncoder;
17
18    // 构造器注入 (Spring 自动装配)
```

业务逻辑层 (III)

在 `com.example.ecommerce.service` 包下创建
`ProductService.java` (商品服务)

```
1package com.example.ecommerce.service;
2
3import com.example.ecommerce.entity.Product;
4import com.example.ecommerce.repository.ProductRepository;
5import org.springframework.stereotype.Service;
6import java.util.List;
7
8@Service
9public class ProductService {
10    private final ProductRepository productRepository;
11
12    public ProductService(ProductRepository productRepository) {
13        this.productRepository = productRepository;
14    }
15
16    // 查询所有商品
17    public List<Product> getAllProducts() {
18        return productRepository.findAll();
19    }
20}
```

业务逻辑层 (IV)

在 `com.example.ecommerce.service` 包下创建
`CartService.java` (购物车服务)

```
1package com.example.ecommerce.service;
2
3import com.example.ecommerce.entity.CartItem;
4import com.example.ecommerce.entity.Product;
5import com.example.ecommerce.entity.User;
6import com.example.ecommerce.repository.CartItemRepository;
7import org.springframework.stereotype.Service;
8import java.util.List;
9
10@Service
11public class CartService {
12    private final CartItemRepository cartItemRepository;
13    private final ProductService productService;
14    private final UserService userService;
15
16    public CartService(CartItemRepository cartItemRepository, ProductService productService,
17                      UserService userService) {
18        this.cartItemRepository = cartItemRepository;
19        this.productService = productService;
20        this.userService = userService;
21    }
22}
```

业务逻辑层 (v)

业务逻辑层解释：

- `@Service`：标记为业务逻辑层
 - `Spring` 会自动管理该类的实例
- 构造器注入：`Spring` 自动将 `Repository` 实例传入
 - 新手无需手动创建对象
- `UserDetailsService`：`Spring Security` 要求的接口
 - 实现 `loadUserByUsername` 即可完成登录认证
- 密码加密：`Spring Security` 强制要求密码加密存储
 - `PasswordEncoder` 会自动处理

配置类

(Config)

(登录、数据库)

配置类

在 `com.example.ecommerce.config` 包下创建
`SecurityConfig.java`（登录配置）

```
1package com.example.ecommerce.config;
2
3import org.springframework.context.annotation.Bean;
4import org.springframework.context.annotation.Configuration;
5import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
7import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
8import org.springframework.security.crypto.password.PasswordEncoder;
9import org.springframework.security.web.SecurityFilterChain;
10
11@Configuration
12@EnableWebSecurity
13public class SecurityConfig {
14    // 密码加密器 (Spring 容器管理)
15    @Bean
16    public PasswordEncoder passwordEncoder() {
17        return new BCryptPasswordEncoder();
18    }
19}
```

安全配置解释：

- `@Configuration`：标记为配置类；
- `PasswordEncoder`：BCrypt 加密器，用于加密用户密码；
- `SecurityFilterChain`：配置登录规则：
- `loginPage("/login")`：
 - 自定义登录页（而非 Spring Security 默认页）；
- `defaultSuccessUrl("/products")`：
 - 登录成功跳转到商品列表；
- `permitAll()`：允许匿名访问登录页，其他页面需登录。

控制器

(Controller)

(接口)

控制器 (I)

- 请求入口与响应
- **角色**：用户请求的 "接收者" 和 "反馈者"，负责衔接前端与后端。
- **作用**：
 - 通过 `@GetMapping`、`@PostMapping` 等注解接收前端请求（如页面访问、表单提交）。
 - 调用服务层方法处理业务，将结果通过 `Model` 传递给前端页面，或直接返回数据。
 - 控制页面跳转（如登录成功后跳转到商品列表，加入购物车后跳回购物车页面）。

控制器 (II)

在 `com.example.ecommerce.controller` 包下创建
`LoginController.java` (登录页)

```
1package com.example.ecommerce.controller;
2
3import org.springframework.stereotype.Controller;
4import org.springframework.web.bind.annotation.GetMapping;
5
6@Controller
7public class LoginController {
8    // 访问 /Login 时返回登录页
9    @GetMapping("/login")
10    public String login() {
11        return "login"; // 对应 templates/login.html
12    }
13}
```

控制器 (III)

在 `com.example.ecommerce.controller` 包下创建
`ProductController.java` (商品列表)

```
1package com.example.ecommerce.controller;
2
3import com.example.ecommerce.service.ProductService;
4import org.springframework.security.core.Authentication;
5import org.springframework.stereotype.Controller;
6import org.springframework.ui.Model;
7import org.springframework.web.bind.annotation.GetMapping;
8
9@Controller
10public class ProductController {
11    private final ProductService productService;
12
13    public ProductController(ProductService productService) {
14        this.productService = productService;
15    }
16
17    // 商品列表页
18    @GetMapping("/products")
```

控制器 (IV)

在 `com.example.ecommerce.controller` 包下创建 `CartController.java` (购物车操作)

```
1package com.example.ecommerce.controller;
2
3import com.example.ecommerce.service.CartService;
4import org.springframework.security.core.Authentication;
5import org.springframework.stereotype.Controller;
6import org.springframework.ui.Model;
7import org.springframework.web.bind.annotation.*;
8
9@Controller
10@RequestMapping("/cart")
11public class CartController {
12    private final CartService cartService;
13
14    public CartController(CartService cartService) {
15        this.cartService = cartService;
16    }
17
18    // 加入购物车
```

控制器解释：

- `@Controller`：标记为 `Web` 控制器，处理前端请求；
- `@GetMapping/@PostMapping`：
 - 处理 `GET/POST` 请求，路径对应前端页面的请求地址；
- `Model`：
 - 用于将数据传递到前端页面（如商品列表、购物车项）；
- `Authentication`：
 - `Spring Security` 提供的对象，获取当前登录用户信息；
- `redirect:/xxx`：
 - 重定向到指定页面（如加入购物车后跳回商品列表）。

前端页面

(基于Thymeleaf)

前端页面 (I)

登录页 ([templates/login.html](#))

```
1<!DOCTYPE html>
2<html xmlns:th="http://www.thymeleaf.org">
3<head>
4    <meta charset="UTF-8">
5    <title>电商网站 - 登录</title>
6    <!-- 引入Bootstrap样式（无需本地下载） -->
7    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/boot
8</head>
9<body class="bg-light">
10    <div class="container mt-5">
11        <div class="row justify-content-center">
12            <div class="col-md-6">
13                <div class="card">
14                    <div class="card-header text-center">
15                        <h3>电商网站登录</h3>
16                    </div>
17                    <div class="card-body">
18                        <!-- 登录表单（提交到 Spring Security 的 /login 接口 -->
```

前端页面 (II)

商品列表页 ([templates/products.html](#))

```
1<!DOCTYPE html>
2<html xmlns:th="http://www.thymeleaf.org">
3<head>
4    <meta charset="UTF-8">
5    <title>商品列表 - 电商网站</title>
6    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/boot
7</head>
8<body>
9    <!-- 导航栏 -->
10    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
11        <div class="container">
12            <a class="navbar-brand" href="/products">电商网站</a>
13            <div class="navbar-nav">
14                <a class="nav-link active" href="/products">商品列表</a>
15                <a class="nav-link" href="/cart">我的购物车</a>
16                <span class="nav-link text-light">欢迎, [[${username}]]</s
17                <a class="nav-link text-light" href="/logout">退出登录</a>
18            </div>
```

前端页面 (III)

购物车页 (templates/cart.html)

```
1<!DOCTYPE html>
2<html xmlns:th="http://www.thymeleaf.org">
3<head>
4    <meta charset="UTF-8">
5    <title>我的购物车 - 电商网站</title>
6    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/boot
7</head>
8<body>
9    <!-- 导航栏 -->
10    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
11        <div class="container">
12            <a class="navbar-brand" href="/products">电商网站</a>
13            <div class="navbar-nav">
14                <a class="nav-link" href="/products">商品列表</a>
15                <a class="nav-link active" href="/cart">我的购物车</a>
16                <span class="nav-link text-light">欢迎, [[${username}]]</s
17                <a class="nav-link text-light" href="/logout">退出登录</a>
18            </div>
```

前端页面解释：

- Thymeleaf 语法：
 - `[[${变量名}]]` 用于显示后端传递的数据
 - `th:each` 用于循环遍历列表
- Bootstrap：通过 CDN 引入，无需本地下载
 - 快速实现美观的页面样式
- 表单提交：`th:action="@{/cart/add}"` 指向控制器的接口
 - 实现加入购物车等操作
- 导航栏：包含商品列表、购物车、退出登录等链接，方便用户操作

Docker

容器化部署与测试

Docker 容器化配置(I)

Dockerfile (Spring Boot 应用容器化)

```
1# 阶段1: 构建 Jar 包 (多阶段构建, 减小镜像体积)
2FROM maven:3.8.5-openjdk-8 AS build
3# 设置工作目录
4WORKDIR /app
5# 复制 pom.xml 和源码
6COPY pom.xml .
7COPY src ./src
8# 打包生成 Jar 包 (跳过测试)
9RUN mvn clean package -DskipTests
10
11# 阶段2: 运行 Jar 包
12FROM openjdk:8-jdk-alpine
13# 设置工作目录
14WORKDIR /app
15# 从构建阶段复制 Jar 包到当前镜像
16COPY --from=build /app/target/ecommerce-1.0.0.jar app.jar
17# 暴露端口 (与 application.properties 中的 server.port 一致)
18EXPOSE 8080
19# 启动命令
```

Docker 容器化配置(II)

Dockerfile 解释：

- 多阶段构建：
 - 先通过 Maven 镜像打包 Jar 包
 - 再用轻量级的 Alpine JDK 镜像运行
 - 减小最终镜像体积
- alpine 版本：
 - 轻量级 Linux 镜像，体积仅几十 MB，比普通 JDK 镜像小很多
- EXPOSE 8080：声明容器暴露的端口
 - 方便 Docker Compose 映射

整合 Spring Boot + MySQL(I)

在项目根目录创建 `docker-compose.yml`:

```
1version: '3.8'
2
3services:
4# MySQL 服务
5mysql:
6    image: mysql:8.0
7    container_name: ecommerce-mysql
8    restart: always
9    environment:
10        # 数据库根密码
11        MYSQL_ROOT_PASSWORD: root123
12        # 自动创建数据库
13        MYSQL_DATABASE: ecommerce_db
14        # 字符集配置
15        MYSQL_CHARACTER_SET_SERVER: utf8mb4
16        MYSQL_COLLATION_SERVER: utf8mb4_unicode_ci
17    ports:
18        - "3306:3306"
19    volumes:
```

整合 Spring Boot + MySQL(II)

`docker-compose.yml` 解释:

- `services`: 定义两个服务: MySQL 和 Spring Boot 应用;
- `mysql` 服务: 使用官方 MySQL 8.0 镜像, 配置根密码、自动创建数据库, 端口映射 3306:3306;
- `app` 服务: 通过 `build:.` 构建当前目录的 `Dockerfile`, 依赖 MySQL 服务 (`depends_on`);
- `networks`: 两个服务在同一网络, Spring Boot 应用可通过 `mysql` 域名访问 MySQL 容器;
- `volumes`: 持久化 MySQL 数据, 避免容器删除后数据丢失。

启动并测试项目(I)

启动容器（项目根目录执行）

```
1# 构建并启动容器（第一次启动加 --build，强制构建镜像）  
2docker compose up --build -d
```

命令解释：

- *up*：启动所有服务
- *--build*：构建镜像（修改代码后需重新构建）
- *-d*：后台运行容器，不占用终端

启动并测试项目(II)

查看容器状态

1# 查看运行中的容器

2docker compose ps

3

4# 查看应用日志（排查错误）

5docker compose logs app

停止容器

1# 停止并删除容器（保留数据卷）

2docker compose down

3

4# 停止并删除容器+数据卷（谨慎：会删除MySQL数据）

5docker compose down -v

功能测试(I)

- 访问登录页：
 - 浏览器输入 <http://localhost:8080/login>;
- 登录：
 - 用户名 [testuser](#)，密码 [123456](#)，点击登录；
- 查看商品列表：
 - 登录成功后跳转到商品列表页，能看到 5 个测试商品；
- 加入购物车：
 - 选择任意商品，输入数量（默认 1），点击“加入购物车”；

功能测试(II)

- 查看购物车：

- 点击导航栏的“我的购物车”，能看到已加入的商品；

- 删除购物车项：

- 点击商品后的“删除”按钮，商品从购物车移除；

- 结算购物车：

- 点击“结算购物车”，购物车清空，跳回商品列表并显示结算成功；

- 退出登录：

- 点击导航栏的“退出登录”，返回登录页。

初始化数据

在 `src/main/resources` 下创建 `data.sql`

```
1-- 初始化测试用户（密码是 123456，已用 BCrypt 加密）
2INSERT INTO users (username, password, full_name)
3VALUES ('testuser', '$2a$10$8H9w5f8G7y6D4s3a2q1z0b9n8m7l6k5j4h3g2f1d0s9a8s7
4
5-- 初始化商品数据
6INSERT INTO products (name, description, price, image_url)
7VALUES
8('小米手机', '高性能智能手机, 8GB+256GB', 2999.00, 'https://picsum.photos/200
9('华为平板', '10.4英寸大屏, 学习办公神器', 1899.00, 'https://picsum.photos/200
10('苹果耳机', '无线蓝牙耳机, 降噪功能', 1299.00, 'https://picsum.photos/200/200
11('联想笔记本', '轻薄本, 16GB+512GB', 4999.00, 'https://picsum.photos/200/200i
12('大疆无人机', '高清航拍, 便携折叠', 3699.00, 'https://picsum.photos/200/200?i
```

数据初始化解释：

- 测试用户密码：123456
- 商品图片用 `picsum.photos` 占位图

课程QQ群



Embedded image 'fig/qqGroup.jpg'